SQL Fundamentals: DDL, DML, Constraints, Aggregates, and NULL Logic

CHAPTER 7 ROADMAP & LEARNING OBJECTIVES

Why This Matters: *Why does referencing a SELECT column alias inside a WHERE clause fail with a syntax error, while referencing it in ORDER BY succeeds? Why does AVG(Marks) disagree with SUM(Marks)/COUNT(*)? And why does NOT IN return zero rows when a single NULL enters a subquery? This chapter gives you mastery over SQL's true 8-stage execution pipeline, DDL/DML engineering, the 7 Deadly NULL Traps, and GROUP BY / HAVING aggregation.*

Prerequisites: Chapters 5 & 6 (Relational Model, Keys, Integrity Constraints, 3VL, and Relational Algebra).

Learning Objectives (What You Will Master in This Chapter):

- Contrast SQL's Lexical (Written) Order with its true 8-Stage Logical Query Evaluation Pipeline (FROM/JOIN -> WHERE -> GROUP BY -> Aggregates -> HAVING -> Window -> SELECT -> DISTINCT -> ORDER BY -> LIMIT/OFFSET).
- Design production SQL DDL using exact financial/temporal data types (why money must use NUMERIC/DECIMAL, never FLOAT), ALTER TABLE evolution, and compare DELETE vs. TRUNCATE vs. DROP across 7 engineering dimensions.
- Write DML queries with LIKE / ESCAPE pattern matching and contrast Set vs. Bag operators (UNION vs. UNION ALL, INTERSECT vs. INTERSECT ALL, EXCEPT vs. EXCEPT ALL) with exact row-count formulas.
- Identify and defeat all 7 Deadly SQL NULL Traps (including COUNT(*) vs. COUNT(col) vs. AVG(col), empty-table aggregate returns, and how UNIQUE, GROUP BY, DISTINCT, and ORDER BY treat multiple NULLs).
- Master GROUP BY, the Single-Value Aggregation Rule, the SQL:1999 Primary Key Functional Dependency Exception, WHERE vs. HAVING, and Conditional Aggregation (CASE inside aggregates & FILTER).

Contents

7 SQL Fundamentals: DDL, DML, Constraints, Aggregates, and NULL Logic 1

7.1 SQL Architecture, Standards, and Logical Execution Order 3

7.1.1 Evolution of ANSI/ISO SQL Standards & The Four SQL Sub-Languages 3

7.1.2 Lexical Order vs. The 8-Stage Logical Query Evaluation Pipeline (Figure 7.1) 4

7.2 Data Definition Language (DDL), Data Types, and Schema Evolution 5

7.2.1 Choosing the Right SQL Data Types (Why Money Must Never Use FLOAT!) 5

7.2.2 Complete Production DDL for UniVerseDB (Listing 7.1) 6

7.2.3 Schema Evolution (ALTER TABLE) & Table 7.1: DELETE vs. TRUNCATE vs. DROP 7

7.3 Data Manipulation Language (DML), Pattern Matching, and Set Operations 8

7.3.1 Table 7.2: Running UniVerseDB Instance (Student & Enrollment with NULLs) 8

7.3.2 Pattern Matching with LIKE and ESCAPE & Set vs. Bag Operations 9

7.4 Three-Valued Logic (3VL), NULL Semantics, and the 7 Deadly SQL Traps 10

7.4.1 Kleene's 3VL & Figure 7.2: The Golden WHERE vs. CHECK Asymmetry 10

7.4.2 The 7 Deadly SQL NULL Traps (Traps 1 to 7 + Table 7.4) 11

7.4.3 Taming NULLs with COALESCE and NULLIF 13

7.5 Aggregation, Grouping (GROUP BY), and Group Filtering (HAVING) 14

7.5.1 Single-Value Rule, SQL:1999 PK Exception & Figure 7.3 Visual Pipeline Trace 14

7.5.2 Conditional Aggregation (CASE / FILTER) & HashAggregate vs. GroupAggregate 16

7.6 Chapter 7 Summary & One-Page Cheat Sheet (Table 7.5) 17

7.7 Chapter 7 Practice Question Bank (Parts A–H) 19

7.8 Complete Answer Key & Worked Solutions for Chapter 7 23

7.1 SQL Architecture, Standards, and Logical Execution Order

Originally developed at IBM Research in the early 1970s as SEQUEL (Structured English Query Language) for the System R prototype, SQL became an ANSI/ISO standard in 1986 and has evolved through SQL-89, SQL-92, SQL:1999 (recursive CTEs, triggers, functional-dependency GROUP BY), SQL:2003 (window functions), SQL:2011 (temporal tables), SQL:2016 (JSON), and SQL:2023 (Property Graph Queries SQL/PGQ).

7.1.1 The Four SQL Sub-Languages

1. DDL (Data Definition Language): CREATE, ALTER, DROP, TRUNCATE, RENAME — Defines and modifies catalog schemas.
2. DML (Data Manipulation Language): SELECT, INSERT, UPDATE, DELETE, MERGE — Queries and modifies relation instances.
3. DCL (Data Control Language): GRANT, REVOKE — Enforces role-based authorization privileges.
4. TCL (Transaction Control Language): BEGIN, COMMIT, ROLLBACK, SAVEPOINT — Controls atomic ACID transaction boundaries.

7.1.2 Lexical (Written) Order vs. Logical Query Evaluation Order

SQL is unique because the order in which you WRITE clauses (SELECT first, FROM second) is NOT the order in which the DBMS logically evaluates them! Understanding the 8-Stage Logical Evaluation Pipeline explains 90% of SQL scoping rules:

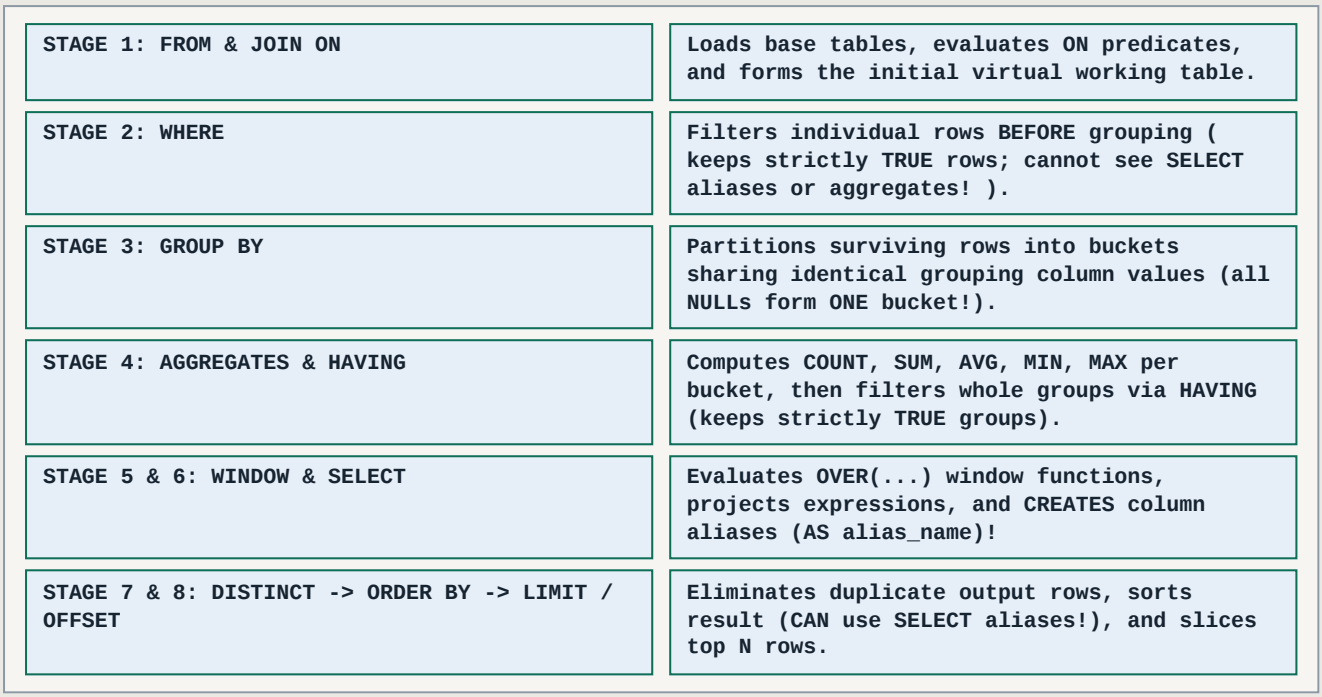


Figure 7.1: The 8-Stage Logical Query Evaluation Pipeline in SQL (Why SELECT Aliases Work in ORDER BY But Fail in WHERE/GROUP BY).

EXAM & INTERVIEW TRAP | Why WHERE Cannot Reference a SELECT Alias or an Aggregate!

Consider: SELECT Name, CGPA * 10 AS Pct FROM Student WHERE Pct > 80; --> ERROR: column "pct" does not exist!
Why? Because Stage 2 (WHERE) executes LONG BEFORE Stage 6 (SELECT), so the alias Pct has not been created yet! Only Stage 7 (ORDER BY Pct DESC) executes after Stage 6 and can legally reference SELECT aliases.
Similarly, WHERE COUNT(*) > 2 is always a syntax error because aggregates are not computed until Stage 4 (use HAVING COUNT(*) > 2)!

7.2 Data Definition Language (DDL), Data Types, and Schema Evolution

7.2.1 Choosing the Right SQL Data Types & Why Money Must NEVER Use FLOAT!

In IEEE 754 binary floating-point (FLOAT / REAL / DOUBLE PRECISION), base-10 fractions like 0.1 and 0.2 have infinite repeating binary representations—so 0.1 + 0.2 evaluates to 0.30000000000000004! In financial or grading databases, rounding drift across 1 million fee receipts loses thousands of rupees.

- Exact Fixed-Point Decimal (NUMERIC(p, s) or DECIMAL(p, s)): Stores exact base-10 digits with precision p (total digits) and scale s (digits after the decimal point). Always use NUMERIC(12, 2) for currency and NUMERIC(4, 2) for CGPA!
- CHAR(n) vs. VARCHAR(n) vs. TEXT: CHAR(n) is fixed-length (right-padded with spaces; use for fixed codes like CHAR(6) Pincode or CHAR(13) ISBN). VARCHAR(n) is variable-length up to n characters (stores only actual length + 1–4 byte header).
- TIMESTAMPTZ vs. TIMESTAMP: Always store event timestamps as TIMESTAMPTZ (TIMESTAMP WITH TIME ZONE, stored internally in UTC) to avoid daylight-saving and multi-region bugs.

Listing 7.1: Production SQL DDL for UniVerseDB with Composite Keys and Named Constraints

```
CREATE TABLE Department (
    DeptID   INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    DeptName VARCHAR(60)  NOT NULL CONSTRAINT uq_dept_name UNIQUE,
    Budget   NUMERIC(14,2) NOT NULL DEFAULT 1000000.00
             CONSTRAINT chk_dept_budget CHECK (Budget >= 0.00)
);

CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    Name      VARCHAR(80) NOT NULL,
    DeptID    INT REFERENCES Department(DeptID) ON DELETE SET NULL,
    CGPA      NUMERIC(4,2) CONSTRAINT chk_cgpa CHECK (CGPA BETWEEN 0.00 AND 10.00),
    Scholar   NUMERIC(10,2) DEFAULT NULL,
    Email     VARCHAR(120) UNIQUE
);
```

7.2.3 Schema Evolution (ALTER TABLE) & Comparing DELETE, TRUNCATE, and DROP

Listing 7.2: Schema Evolution with ALTER TABLE (Adding, Modifying, and Dropping Constraints)

```
-- 1. Add a new column with a DEFAULT value
ALTER TABLE Student ADD COLUMN AdmissionYear SMALLINT NOT NULL DEFAULT 2026;

-- 2. Add a named CHECK constraint to an existing table
ALTER TABLE Student ADD CONSTRAINT chk_adm_year CHECK (AdmissionYear >= 2000);

-- 3. Drop a constraint or column
ALTER TABLE Student DROP CONSTRAINT chk_adm_year;
ALTER TABLE Student DROP COLUMN AdmissionYear;
```

Table 7.1: Complete 7-Dimension Engineering Comparison of DELETE, TRUNCATE, and DROP

Dimension	DELETE FROM tbl [WHERE ...]	TRUNCATE TABLE tbl	DROP TABLE tbl
1. SQL Sub-Language	DML (Data Manipulation Language)	DDL (Data Definition Language)	DDL (Data Definition Language)
2. What It Removes	Selected rows (or all rows if no WHERE); Schema stays intact.	All rows (resets table to Empty State); Schema stays intact.	Removes ALL rows, indexes, triggers AND the Schema from Catalog!
3. Supports WHERE Clause?	YES (Row-by-row filtering)	NO (Always removes all rows)	NO (Destroys entire table)
4. Execution Speed & WAL Logging	Slowest on large tables (logs every deleted row individually in WAL).	Ultra-Fast O(1) (deallocates data pages/files in bulk).	Ultra-Fast O(1) (deallocates files + removes catalog metadata).
5. Fires Row-Level Triggers?	YES (Fires BEFORE/AFTER DELETE triggers per row).	NO (Does not fire row-level ON DELETE triggers).	NO (Drops triggers along with the table).
6. Space Reclamation & Identity	Leaves dead tuples (needs VACUUM in PG); keeps sequence counter.	Reclaims disk pages immediately; optional RESTART IDENTITY.	Reclaims all disk files and drops owned sequences/indexes.
7. Can Be Rolled Back (ACID)?	YES (Inside an uncommitted BEGIN ... ROLLBACK transaction).	YES in PostgreSQL & SQL Server; auto-commits in MySQL/Oracle.	YES in PostgreSQL & SQL Server; auto-commits in MySQL/Oracle.

7.3 Data Manipulation Language (DML), Pattern Matching, and Set Operations

To trace every query, NULL trap, and GROUP BY calculation in this chapter with 100% precision, we use the following 6-row Student instance and 7-row Enrollment instance:

Table 7.2: Running UniVerseDB Instance — Student (6 Rows) and Enrollment (7 Rows) with Strategic NULLs

StudentID	Name	DeptID	CGPA	Scholar (Rs.)	Enrollment Rows (StudentID, CourseID, Marks, Grade)
1	Asha	10	8.50	50000	(1, "CS301", 90, "A+"), (1, "CS201", 80, "A")
2	Ravi	10	7.50	NULL	(2, "CS301", 70, "B"), (2, "CS201", NULL, NULL) <-- Exam pending!
3	Meera	20	9.00	60000	(3, "EC204", 95, "A+"), (3, "CS301", 85, "A")
4	Kabir	NULL	6.50	NULL	No rows in Enrollment (0 enrollments)
5	Divya	20	8.00	40000	(5, "EC204", 75, "B")
6	Rohan	10	NULL	10000	No rows in Enrollment (First-semester transfer, CGPA = NULL)

7.3.2 Pattern Matching with LIKE and ESCAPE

- Percent Wildcard (%): Matches ANY sequence of zero or more characters (e.g., LIKE 'A%' matches "A", "Asha", "Ananya").
- Underscore Wildcard (_): Matches EXACTLY ONE character (e.g., LIKE '____' (4 underscores) matches "Asha" and "Ravi", but not "Meera").
- Matching Literal % or _ with ESCAPE: How do you find strings containing a literal underscore "_" or "100%"? Specify an escape character:
  WHERE PromoCode LIKE '100\%%' ESCAPE '\' (Matches strings starting with literal "100%").
  WHERE TableName LIKE 'stu_%_v1' ESCAPE '\' (Matches "stu_" followed by anything and ending in "_v1").

7.3.3 Set vs. Bag Operations in SQL: UNION, INTERSECT, and EXCEPT

Suppose query Q1 returns a bag of m copies of tuple t, and query Q2 returns a bag of n copies of tuple t. How many copies of tuple t appear in the output of SQL's Set vs. Bag operators?

Exact Multiplicity Formulas for SQL Set Operations vs. Bag (ALL) Operations (Given m copies in Q1, n copies in Q2)

SQL Operator	Semantics	Exact Copies of Tuple t in Output	Example (m = 3 in Q1, n = 2 in Q2)
Q1 UNION Q2	Set Union (Deduplicated)	1 (if m > 0 or n > 0)	1 copy of t
Q1 UNION ALL Q2	Bag Union (Concatenation)	m + n	3 + 2 = 5 copies of t (Zero sort/hash overhead!)
Q1 INTERSECT Q2	Set Intersection	1 (if m > 0 and n > 0, else 0)	1 copy of t
Q1 INTERSECT ALL Q2	Bag Intersection	min(m, n)	min(3, 2) = 2 copies of t
Q1 EXCEPT Q2 (or MINUS)	Set Difference	1 (if m > 0 and n = 0, else 0)	0 copies (since n = 2 > 0 in Q2!)
Q1 EXCEPT ALL Q2	Bag Difference	max(0, m − n)	max(0, 3 − 2) = 1 copy of t survives!

GATE NAT FAVORITE | Counting Duplicates in EXCEPT vs. EXCEPT ALL

If Q1 returns (10, 10, 10, 20, 20) and Q2 returns (10, 20, 20, 20):
• Q1 EXCEPT Q2 returns 0 rows (both 10 and 20 appear in Q2).
• Q1 EXCEPT ALL Q2 returns (10, 10) — 2 rows! Because for value 10: max(0, 3 − 1) = 2 copies; for value 20: max(0, 2 − 3) = 0 copies!

7.4 Three-Valued Logic (3VL), NULL Semantics, and the 7 Deadly SQL Traps

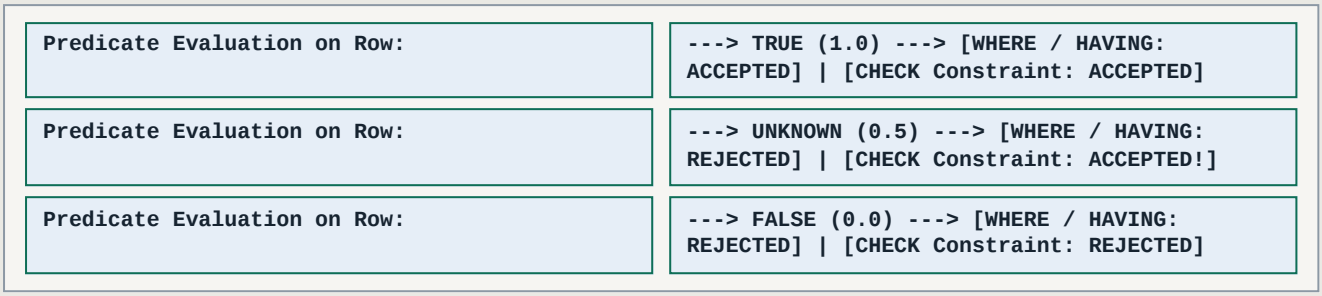


Figure 7.2: The Golden Asymmetry Between WHERE / HAVING / ON (Passes Strictly TRUE) and CHECK Constraints (Passes TRUE or UNKNOWN).

7.4.1 The 7 Deadly SQL NULL Traps (Traps 1 to 4)

1. Trap 1 — Comparing with = NULL or <> NULL: Writing WHERE Scholar = NULL or WHERE Scholar <> NULL evaluates to UNKNOWN for every row and returns 0 rows! Always use IS NULL, IS NOT NULL, or IS [NOT] DISTINCT FROM.
2. Trap 2 — The NOT IN (Subquery with NULL) Disaster: x NOT IN (10, 20, NULL) expands to (x <> 10) AND (x <> 20) AND (x <> NULL). Because (x <> NULL) is UNKNOWN, the entire AND expression is either FALSE or UNKNOWN—never TRUE! Result: 0 rows returned! (Whereas x IN (10, 20, NULL) expands with OR, so x = 10 evaluates to TRUE OR FALSE OR UNKNOWN = TRUE!).
3. Trap 3 — Failure of the Law of the Excluded Middle: On Table 7.2, SELECT COUNT(*) FROM Student WHERE Scholar > 20000 OR Scholar <= 20000 returns only 4 rows (Asha, Meera, Divya, Rohan), silently dropping Ravi and Kabir whose Scholar is NULL!
4. Trap 4 — The Aggregate Asymmetry: COUNT(*) vs. COUNT(col) vs. AVG(col): Every SQL aggregate function EXCEPT COUNT(*) completely ignores NULL values before computing!

Table 7.4: Tracing Aggregate Functions on Table 7.2 Student (6 Rows: Scholar = {50000, NULL, 60000, NULL, 40000, 10000})

Aggregate Expression on Student	How NULLs Are Handled	Exact Value on Table 7.2 (6 Rows)
COUNT(*)	Counts ALL rows regardless of NULLs.	6
COUNT(Scholar)	Ignores the 2 NULL rows (Ravi, Kabir); counts non-null values.	4
COUNT(DISTINCT DeptID)	Ignores NULL (Kabir), then deduplicates {10,10,20,20,10} -> {10, 20}.	2
SUM(Scholar)	Sums the 4 non-null values: 50000 + 60000 + 40000 + 10000.	160000
AVG(Scholar)	Divides SUM(Scholar) by COUNT(Scholar) (4, NOT 6!): 160000 / 4.	40000.00 (NOT 160000/6 = 26666.67!)
AVG(COALESCE(Scholar, 0))	Converts NULLs to 0 first, dividing by all 6 students: 160000 / 6.	26666.67

7.4.2 The 7 Deadly SQL NULL Traps (Traps 5, 6, and 7)

- 5. Trap 5 — Aggregate Functions on an Empty Set (0 Matching Rows): If WHERE 1 = 0 matches 0 rows (and there is no GROUP BY), what do aggregates return?
  • COUNT(*) and COUNT(col) return 0.
  • SUM(col), AVG(col), MIN(col), and MAX(col) all return NULL (NOT 0)!
  (Wait: what if there IS a GROUP BY clause and the WHERE clause matches 0 rows? Then the query returns 0 rows total, because 0 groups are formed!).
- 6. Trap 6 — Contagious NULLs in Scalar Arithmetic (+, −, *, /) vs. Aggregate SUM():
  • Scalar row arithmetic: 50000 + NULL evaluates to NULL! (In standard SQL, 'Asha' || NULL also evaluates to NULL).
  • Aggregate vertical sum: SUM() ignores NULL rows and sums the remaining numbers!
- 7. Trap 7 — Where SQL Treats Multiple NULLs as EQUAL vs. DISTINCT:
  • In WHERE / JOIN ON: NULL = NULL is UNKNOWN (do NOT match).
  • In UNIQUE constraints: Multiple NULLs are treated as DISTINCT (allowed).
  • In GROUP BY, SELECT DISTINCT, UNION/INTERSECT/EXCEPT, and ORDER BY: All NULLs are grouped together into ONE single bucket!

7.4.3 Taming NULLs with COALESCE and NULLIF

Defensive SQL Patterns Using COALESCE and NULLIF

```
-- 1. COALESCE(v1, v2, ..., vn) returns the FIRST non-null argument:
SELECT Name,
       COALESCE(Scholar, 0) AS EffectiveScholar,
       5000 + COALESCE(Scholar, 0) AS TotalGrant  -- Prevents 5000 + NULL = NULL!
FROM Student;

-- 2. NULLIF(a, b) returns NULL if a = b, otherwise returns a (prevents Division-by-Zero!):
SELECT CourseID,
       PassedCount * 100.0 / NULLIF(TotalEnrolled, 0) AS PassPercentage
FROM Course_Stats;
```

7.5 Aggregation, Grouping (GROUP BY), and Group Filtering (HAVING)

7.5.1 The Single-Value Rule & The SQL:1999 Primary Key Exception

1. The Single-Value Rule of GROUP BY: When a query contains a GROUP BY clause (or an aggregate function), every unaggregated column appearing in the SELECT or HAVING clause MUST also appear in the GROUP BY clause! Why? Because there is only one output row per group—if you group by DeptID (which has 3 students: Asha, Ravi, Rohan) and write SELECT DeptID, Name, COUNT(*), the DBMS cannot pick which of the 3 names to display!
2. The SQL:1999 Functional Dependency Exception (Supported by PostgreSQL & MySQL): If you group by the PRIMARY KEY of a table (e.g., GROUP BY s.StudentID), you ARE allowed to select s.Name and s.CGPA in the SELECT clause without listing them in GROUP BY! Why? Because StudentID -> {Name, CGPA} is a functional dependency (each StudentID uniquely determines a single Name and CGPA).

7.5.2 Step-by-Step Visual Trace of WHERE vs. GROUP BY vs. HAVING

Listing 7.3: Tracing WHERE, GROUP BY, and HAVING on Table 7.2 Student

```
SELECT DeptID,
       COUNT(*)        AS TotalStudents,
       COUNT(CGPA)     AS GradedStudents,
       AVG(CGPA)       AS AvgCGPA
FROM Student
WHERE Scholar IS NULL OR Scholar >= 10000
GROUP BY DeptID
HAVING COUNT(*) >= 1
ORDER BY AvgCGPA DESC NULLS LAST;
```

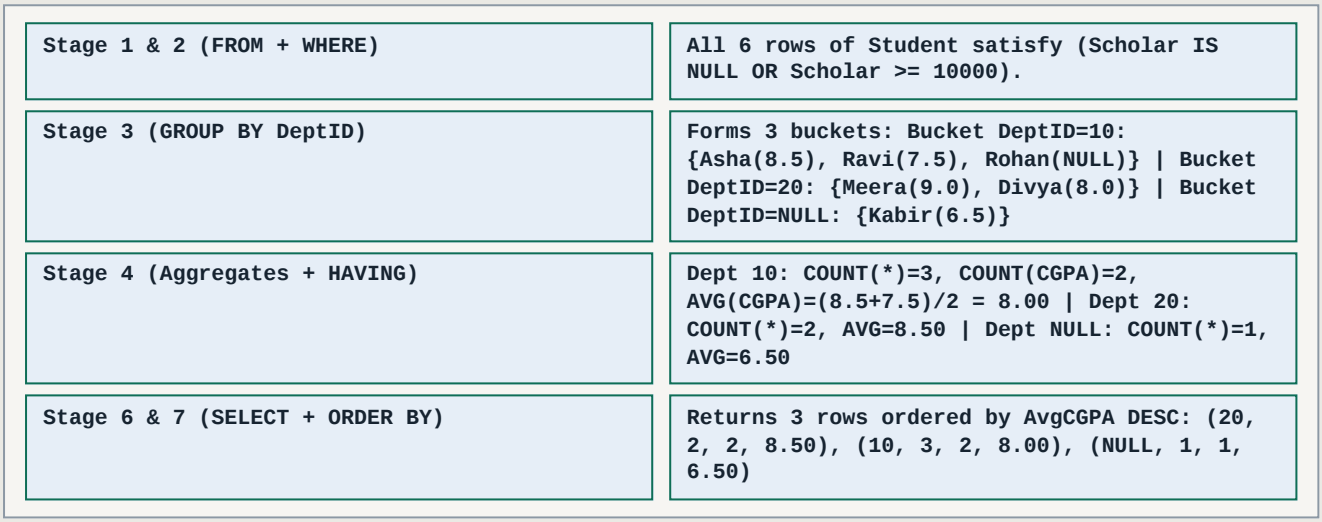


Figure 7.3: Step-by-Step Execution Trace of Listing 7.3 on Table 7.2 (Showing the NULL Group Bucket!).

7.5.3 Conditional Aggregation (CASE Inside Aggregates & SQL:2003 FILTER)

In placement interviews and reporting dashboards, you often need to pivot multiple filtered counts or sums in a single table scan (e.g., counting Honours students vs. Scholarship students per department without running 3 separate subqueries):

Listing 7.4: Conditional Aggregation Using CASE WHEN Inside SUM/COUNT and PostgreSQL FILTER

```
SELECT DeptID,
       COUNT(*) AS TotalStudents,
       -- Pattern 1: SUM(CASE WHEN ... THEN 1 ELSE 0 END)
       SUM(CASE WHEN CGPA >= 8.50 THEN 1 ELSE 0 END) AS DistinctionCount,
       -- Pattern 2: COUNT(CASE WHEN ... THEN 1 END)  [ELSE NULL is implicit and ignored by COUNT!]
       COUNT(CASE WHEN Scholar IS NOT NULL THEN 1 END) AS ScholarshipHolders,
       -- Pattern 3: Modern SQL:2003 / PostgreSQL FILTER clause
       AVG(CGPA) FILTER (WHERE Scholar IS NOT NULL) AS AvgScholarCGPA
FROM Student
GROUP BY DeptID;
```

INTERVIEW TRAP | Why COUNT(CASE WHEN cond THEN 1 ELSE 0 END) Is a Bug!

In SQL, COUNT(expr) counts every row where expr is NOT NULL—including 0 and FALSE! Therefore, COUNT(CASE WHEN CGPA >= 8.5 THEN 1 ELSE 0 END) counts the 0s too and returns COUNT(*)! Either use SUM(CASE WHEN ... THEN 1 ELSE 0 END) OR omit ELSE 0 so unmatched rows default to NULL: COUNT(CASE WHEN CGPA >= 8.5 THEN 1 END)!

7.5.4 DBMS Engine Internals: HashAggregate vs. GroupAggregate

1. HashAggregate (O(N) In-Memory Hash Table): The Execution Engine scans the input rows once, hashes the GROUP BY key (DeptID), and updates running accumulators (count, sum) in an in-memory hash bucket. Fast when the number of distinct groups fits in work_mem.
2. GroupAggregate / SortAggregate (O(N log N) Sort or O(N) via B+ Tree Index): First sorts rows by the GROUP BY columns (or reads them in order from an existing B+ Tree index) and emits one aggregate row each time the group key changes. Uses almost zero RAM!

7.6 Chapter 7 Summary & One-Page Cheat Sheet

Table 7.5: Chapter 7 Master SQL Cheat Sheet

SQL Feature / Clause	Core Execution Rule & Exam Trap
Logical Execution Order	1. FROM/JOIN -> 2. WHERE -> 3. GROUP BY -> 4. Aggregates/HAVING -> 5. Window -> 6. SELECT -> 7. DISTINCT -> 8. ORDER BY -> 9. LIMIT.
SELECT Alias Scope	Created at Stage 6 (SELECT). Invisible in WHERE, GROUP BY, and HAVING; visible in ORDER BY!
DELETE vs. TRUNCATE vs. DROP	DELETE = DML (row-logged, WHERE allowed, triggers fire). TRUNCATE = DDL (bulk page dealloc, keeps schema). DROP = DDL (destroys table & schema).
Set vs. Bag Multiplicities	UNION ALL = m+n; INTERSECT ALL = min(m,n); EXCEPT ALL = max(0, m−n). Plain UNION/INTERSECT/EXCEPT deduplicate.
COUNT(*) vs. COUNT(col) vs. AVG	COUNT(*) counts all rows including NULLs; COUNT(col), SUM(col), AVG(col) ignore NULLs! AVG(col) = SUM(col)/COUNT(col) ≠ SUM(col)/COUNT(*).
Aggregates on 0 Rows (No GROUP BY)	COUNT returns 0; SUM, AVG, MIN, MAX all return NULL!
How SQL Groups NULLs	WHERE/JOIN treat NULL=NULL as UNKNOWN; GROUP BY, DISTINCT, and Set ops group all NULLs into ONE bucket!
WHERE vs. HAVING	WHERE filters individual rows BEFORE GROUP BY (no aggregates allowed). HAVING filters groups AFTER GROUP BY.

Student Revision & Lecture Notes — Chapter 7

Use this revision page to practice the 8-stage execution order, the 7 Deadly NULL Traps, and tracing GROUP BY / HAVING queries on Table 7.2.

Active Recall Checklist — Chapter 7

Prompt	Your Recall Answer
Why must financial amounts use NUMERIC(12,2) instead of FLOAT?	IEEE 754 binary floating-point cannot represent base-10 decimals like 0.10 exactly, causing rounding errors.
What does SELECT SUM(Marks), COUNT(*) FROM Enrollment WHERE 1=0 return?	Returns 1 row: (NULL, 0).
How many groups does GROUP BY DeptID form on {10, 10, 20, NULL, NULL}?	3 groups: Dept 10, Dept 20, and one combined group for all NULLs!
Fix COUNT(CASE WHEN Grade="A" THEN 1 ELSE 0 END)	Use SUM(CASE WHEN Grade="A" THEN 1 ELSE 0 END) or COUNT(CASE WHEN Grade="A" THEN 1 END).

7.7 Chapter 7 Practice Question Bank

Part A: 10 Multiple-Choice Questions (MCQs)

1. Q1. In what logical order does an SQL engine evaluate the following clauses of a query?
   1. SELECT 2. WHERE 3. FROM 4. GROUP BY 5. HAVING 6. ORDER BY
   (A) 1 -> 3 -> 2 -> 4 -> 5 -> 6 (B) 3 -> 2 -> 4 -> 5 -> 1 -> 6 (C) 3 -> 4 -> 2 -> 5 -> 1 -> 6 (D) 2 -> 3 -> 4 -> 5 -> 6 -> 1
2. Q2. On the 6-row Student table in Table 7.2 (where Scholar values are 50000, NULL, 60000, NULL, 40000, 10000), what does SELECT COUNT(*), COUNT(Scholar), AVG(Scholar) FROM Student; return?
   (A) 6, 6, 26666.67 (B) 6, 4, 40000.00 (C) 4, 4, 40000.00 (D) 6, 4, NULL
3. Q3. Which SQL pattern matches any string that contains at least 3 characters and ends with the letter "a"?
   (A) LIKE '___a' (B) LIKE '__%a' (C) LIKE '%___a%' (D) LIKE '3%a'
4. Q4. What is returned by SELECT COUNT(*), SUM(CGPA) FROM Student WHERE 1 = 2; (assuming Student has 6 rows)?
   (A) 0 rows returned (B) 1 row: (0, 0) (C) 1 row: (0, NULL) (D) 1 row: (NULL, NULL)
5. Q5. Suppose Query A returns the bag {5, 5, 5, 8, 8} and Query B returns the bag {5, 8, 8, 8}. How many total rows are returned by (Query A) EXCEPT ALL (Query B)?
   (A) 0 (B) 1 (C) 2 (D) 3

Part A: Multiple-Choice Questions (Continued — Q6 to Q10)

- 6. Q6. Which of the following SQL queries will fail with a compilation/syntax error in standard SQL?
  (A) SELECT DeptID, COUNT(*) FROM Student GROUP BY DeptID HAVING COUNT(*) > 1;
  (B) SELECT Name, CGPA * 10 AS Score FROM Student ORDER BY Score DESC;
  (C) SELECT DeptID, AVG(CGPA) FROM Student WHERE AVG(CGPA) > 8.0 GROUP BY DeptID;
  (D) SELECT DeptID, MAX(CGPA) FROM Student WHERE CGPA IS NOT NULL GROUP BY DeptID;
- 7. Q7. On Table 7.2 Student (where DeptID values are 10, 10, 20, NULL, 20, 10), how many output rows are returned by SELECT DeptID, COUNT(*) FROM Student GROUP BY DeptID;?
  (A) 2 rows (B) 3 rows (C) 4 rows (D) 6 rows
- 8. Q8. What is the output of SELECT COALESCE(NULL, NULLIF(10, 10), 25, 40);?
  (A) NULL (B) 10 (C) 25 (D) 40
- 9. Q9. Which statement accurately contrasts DELETE FROM Student; and TRUNCATE TABLE Student;?
  (A) TRUNCATE fires row-level ON DELETE triggers whereas DELETE does not
  (B) DELETE allows a WHERE clause and logs individual row deletions, whereas TRUNCATE removes all rows by deallocating data pages
  (C) TRUNCATE drops the Student schema from the System Catalog
  (D) DELETE resets the column data types
- 10. Q10. What does SELECT COUNT(CASE WHEN CGPA >= 8.50 THEN 1 ELSE 0 END) FROM Student; compute on the 6-row Student table?
  (A) The number of students with CGPA >= 8.50 (2) (B) The total number of rows in Student (6), because both 1 and 0 are non-NULL (C) 0 (D) NULL

Part B: 5 Short-Answer Questions (2–3 Marks Semester Exam Style)

1. B1. Explain why a column alias defined in the SELECT clause cannot be used in the WHERE or GROUP BY clause, but CAN be used in the ORDER BY clause.
2. B2. Compare DELETE, TRUNCATE, and DROP across DML/DDL classification, WHERE filtering, trigger execution, and catalog schema impact.
3. B3. Why is AVG(Col) generally NOT equal to SUM(Col) / COUNT(*) when Col contains NULL values, and how can you write an expression that treats NULLs as 0 in the average?
4. B4. State the Single-Value Rule of GROUP BY and explain the SQL:1999 Primary Key Functional Dependency exception.
5. B5. Explain how SQL treats multiple NULL values in (a) a WHERE equality join, (b) a UNIQUE constraint, and (c) a GROUP BY clause.

Part C: 3 Long-Answer / Essay Questions (5–10 Marks Semester Exam Style)

1. C1. Draw and explain the 8-Stage Logical Query Evaluation Pipeline in SQL. Using a concrete multi-clause query on UniVerseDB, trace how the working table transforms at each stage from FROM to LIMIT.
2. C2. Present and explain the 7 Deadly SQL NULL Traps with concrete SQL examples and outputs on the Student and Enrollment tables, and show the defensive SQL pattern that fixes each trap.
3. C3. Compare WHERE and HAVING in detail. Explain when predicates can be pushed from HAVING into WHERE for query optimization, and demonstrate Conditional Aggregation using CASE expressions inside SUM/COUNT.

Part D: 4 Query Output Trace & Debugging Problems

1. D1. (Exact Output Trace on Table 7.2): Trace the exact output rows and columns of the following 3 queries on Table 7.2:
   (a) SELECT DeptID, COUNT(*), COUNT(Scholar), SUM(Scholar) FROM Student GROUP BY DeptID;
   (b) SELECT CourseID, COUNT(*), COUNT(Marks), AVG(Marks) FROM Enrollment GROUP BY CourseID;
   (c) SELECT Name FROM Student WHERE DeptID NOT IN (SELECT DeptID FROM Student WHERE CGPA < 7.00);
2. D2. (Bag Multiplicity Calculation): Let Table R(X) contain {1, 1, 1, 2, 2, 3} (6 rows) and Table S(X) contain {1, 2, 2, 2, 4} (5 rows). Compute the exact multiset of rows and total row count for: (i) R UNION S, (ii) R UNION ALL S, (iii) R INTERSECT S, (iv) R INTERSECT ALL S, (v) R EXCEPT S, and (vi) R EXCEPT ALL S.
3. D3. (Pattern Matching with ESCAPE): Write a SQL WHERE predicate using LIKE and ESCAPE to match strings in column Code that: (a) start with literal "CS_", followed by exactly 2 characters, and end with literal "%"; (b) contain at least two "a" characters anywhere.
4. D4. (Conditional Aggregation Report): Write a single-scan SQL query on Enrollment(StudentID, CourseID, Marks, Grade) that outputs per CourseID: TotalEnrolled, PassedCount (Grade <> "F" and Grade IS NOT NULL), PendingExamCount (Marks IS NULL), and PassRatePct.

Part E: 5 GATE CS Style Questions (MCQ, MSQ, and NAT)

1. E1. [GATE NAT — 2 Marks] Consider a table T(A INT, B INT) containing the following 5 rows:
   (1, 10), (1, NULL), (2, 20), (2, 30), (NULL, 40).
   When the query SELECT A, COUNT(B), SUM(B) FROM T GROUP BY A HAVING COUNT(*) >= 1; is executed, the sum of the values in the third column (SUM(B)) across all output rows is _____, and the number of output rows is _____.
2. E2. [GATE NAT — 2 Marks] Let R(A) have 4 copies of value 10 and 2 copies of value 20. Let S(A) have 1 copy of value 10 and 5 copies of value 20. The number of rows returned by the SQL query:
   (SELECT A FROM R INTERSECT ALL SELECT A FROM S) UNION ALL (SELECT A FROM R EXCEPT ALL SELECT A FROM S)
   is _____.
3. E3. [GATE MCQ — 2 Marks] Consider the following SQL query on Student(_StudentID_, Name, DeptID, CGPA):
   SELECT DeptID, AVG(CGPA) FROM Student WHERE CGPA >= 7.0 GROUP BY DeptID HAVING COUNT(*) >= 2;
   Which sequence of operations correctly describes its logical evaluation?
   (A) Group by DeptID -> Filter groups with COUNT(*) >= 2 -> Filter rows with CGPA >= 7.0 -> Compute AVG(CGPA)
   (B) Filter rows with CGPA >= 7.0 -> Group surviving rows by DeptID -> Keep groups with at least 2 surviving rows -> Project DeptID and AVG(CGPA)
   (C) Compute AVG(CGPA) over entire table -> Filter by WHERE -> Group by DeptID
   (D) Filter groups by HAVING -> Filter rows by WHERE -> Group by DeptID
4. E4. [GATE MSQ — 2 Marks] Which of the following SQL queries on Student(StudentID PK, Name, DeptID, CGPA) are VALID in SQL:1999 / PostgreSQL?
   (A) SELECT StudentID, Name, COUNT(*) FROM Student GROUP BY StudentID;
   (B) SELECT DeptID, Name, COUNT(*) FROM Student GROUP BY DeptID;
   (C) SELECT DeptID, COUNT(*) AS Cnt FROM Student GROUP BY DeptID ORDER BY Cnt DESC;
   (D) SELECT DeptID, COUNT(*) FROM Student GROUP BY DeptID HAVING MAX(CGPA) > 8.5;
5. E5. [GATE NAT — 2 Marks] On the 6-row Student table in Table 7.2 (CGPAs: 8.50, 7.50, 9.00, 6.50, 8.00, NULL), what numerical value is returned by:
   SELECT AVG(COALESCE(CGPA, 6.50)) - AVG(CGPA) FROM Student; (Round to 2 decimal places): _____.

Part F: 5 Viva / Technical Placement Interview Questions

1. F1. "What is the difference between WHERE and HAVING, and can a condition ever be written in either clause?"
2. F2. "In a production PostgreSQL database with 50 million rows, why is TRUNCATE TABLE instant while DELETE FROM tbl takes 4 minutes and bloats disk space?"
3. F3. "Why should you always prefer UNION ALL over UNION when you know the two input sets are disjoint (or when duplicates don’t matter)?"
4. F4. "What happens when a Python (psycopg2) or Java (JDBC) backend fetches a SQL NULL value—and what is the classic JDBC getInt() trap?"
5. F5. "Explain how GROUP BY and HAVING work together when finding duplicate emails in a Users table."

Part G: 3 Hands-On SQL Labs & Part H: 3 "Spot the Mistake" Challenges

1. Lab 7.1: Write a SQL query to find all duplicate (Name, City) pairs in a table and keep only the lowest StudentID.
2. Lab 7.2: Write a Department KPI dashboard query using GROUP BY, COALESCE, and CASE WHEN / FILTER.
3. Lab 7.3: Verify the Empty-Set Aggregate trap by running SELECT COUNT(*), SUM(CGPA), COALESCE(SUM(CGPA), 0) on an empty filter.
4. Mistake 7.1: A developer writes SELECT DeptID, AVG(CGPA) AS AvgG FROM Student WHERE AvgG > 8.0 GROUP BY DeptID;.
5. Mistake 7.2: To calculate total compensation (Salary + Bonus) where Bonus can be NULL, a developer writes SELECT Name, Salary + Bonus AS TotalComp FROM Employee;.
6. Mistake 7.3: A developer writes SELECT DeptID, Name, AVG(CGPA) FROM Student GROUP BY DeptID; expecting it to return the name of the student with the highest CGPA in each department.

7.8 Complete Answer Key & Worked Solutions for Chapter 7

Solutions to Part A: Multiple-Choice Questions (Q1 to Q10)

Answer Key & Detailed Explanations for Part A MCQs

Q#	Ans	Detailed Step-by-Step Explanation
Q1	(B)	Logical order: FROM (3) -> WHERE (2) -> GROUP BY (4) -> HAVING (5) -> SELECT (1) -> ORDER BY (6).
Q2	(B)	COUNT(*) = 6 rows; COUNT(Scholar) = 4 non-null rows; AVG(Scholar) = 160000 / 4 = 40000.00.
Q3	(B)	Two underscores (__) require at least 2 characters + % (0 or more) + "a" = at least 3 characters ending in "a".
Q4	(C)	On 0 matching rows without GROUP BY, COUNT(*) returns 0 and SUM(CGPA) returns NULL -> 1 row: (0, NULL).
Q5	(C)	In EXCEPT ALL: for 5, max(0, 3 − 1) = 2 copies; for 8, max(0, 2 − 3) = 0 copies -> 2 rows total!
Q6	(C)	Aggregate functions like AVG(CGPA) are never allowed inside a WHERE clause (must be in HAVING).
Q7	(B)	GROUP BY DeptID forms 3 groups: DeptID = 10, DeptID = 20, and DeptID = NULL (all NULLs form 1 bucket!).
Q8	(C)	NULLIF(10, 10) evaluates to NULL; COALESCE(NULL, NULL, 25, 40) returns the first non-null value: 25.
Q9	(B)	DELETE is row-logged DML supporting WHERE and triggers; TRUNCATE is bulk page-deallocating DDL without WHERE.
Q10	(B)	COUNT(expr) counts every non-null value. Since CASE returns either 1 or 0 (neither is NULL!), it counts all 6 rows!

Solutions to Part B: Short-Answer Questions (B1 to B5)

- Answer B1 (SELECT Alias Scope): In the 8-stage logical execution pipeline, WHERE (Stage 2) and GROUP BY (Stage 3) are evaluated BEFORE the SELECT projection list (Stage 6), so aliases created in SELECT do not exist yet. ORDER BY (Stage 8) executes AFTER SELECT, so it can reference SELECT aliases.
- Answer B2 (DELETE vs. TRUNCATE vs. DROP): DELETE is DML, supports WHERE, fires row triggers, logs each deleted row, and keeps the schema. TRUNCATE is DDL, removes all rows by deallocating data pages in O(1), does not fire row triggers, and keeps the schema. DROP is DDL and destroys both data files and the catalog schema.
- Answer B3 (Why AVG(Col) ≠ SUM(Col)/COUNT(*)): AVG(Col) ignores NULL rows and divides SUM(Col) by COUNT(Col) (the number of non-null rows). To treat NULLs as 0 across all rows, write AVG(COALESCE(Col, 0)) or SUM(Col) / COUNT(*).
- Answer B4 (Single-Value Rule & PK Exception): Every unaggregated column in SELECT/HAVING must appear in GROUP BY so it has a single well-defined value per group. Under SQL:1999, if you group by a table's Primary Key, functionally dependent columns of that same table may appear in SELECT without being listed in GROUP BY.
- Answer B5 (How SQL Treats Multiple NULLs): (a) In WHERE/JOIN equality (a = b), NULL = NULL is UNKNOWN (rows do not match). (b) In a UNIQUE constraint, multiple NULLs are treated as distinct (allowed). (c) In GROUP BY, DISTINCT, and UNION, all NULLs are grouped together into a single bucket.

Solutions to Part C: Long-Answer / Essay Questions (C1 to C3)

Model Solutions C1, C2 & C3 | Execution Pipeline, 7 NULL Traps & WHERE vs. HAVING Pushdown

• Model Solution C1 (8-Stage Pipeline): Walk through Stages 1 to 8 (FROM/JOIN -> WHERE -> GROUP BY -> Aggregates/HAVING -> Window -> SELECT -> DISTINCT -> ORDER BY -> LIMIT), illustrating each stage on Listing 7.3.
• Model Solution C2 (7 NULL Traps): Present Traps 1–7 (= NULL, NOT IN with NULL, Excluded Middle failure, COUNT(*) vs. AVG(col), Empty-Set aggregates returning NULL, Contagious arithmetic NULLs, and GROUP BY/DISTINCT bucketing NULLs) along with COALESCE/NOT EXISTS fixes.
• Model Solution C3 (WHERE vs. HAVING Optimization): Predicates that filter only on grouping columns (e.g., HAVING DeptID = 10) can and should be pushed into WHERE DeptID = 10 so rows are discarded BEFORE sorting/hashing for GROUP BY. Predicates involving aggregates (HAVING AVG(CGPA) > 8.0) cannot be pushed to WHERE.

Solutions to Part D: Query Output Trace & Debugging Problems (D1 & D2)

Worked Solution D1 | Exact Output Traces on Table 7.2

(a) SELECT DeptID, COUNT(*), COUNT(Scholar), SUM(Scholar) FROM Student GROUP BY DeptID;
• DeptID = 10: COUNT(*) = 3, COUNT(Scholar) = 2, SUM(Scholar) = 60000
• DeptID = 20: COUNT(*) = 2, COUNT(Scholar) = 2, SUM(Scholar) = 100000
• DeptID = NULL: COUNT(*) = 1, COUNT(Scholar) = 0, SUM(Scholar) = NULL <-- All NULLs in group => SUM is NULL!

(b) SELECT CourseID, COUNT(*), COUNT(Marks), AVG(Marks) FROM Enrollment GROUP BY CourseID;
• "CS301": COUNT(*) = 3, COUNT(Marks) = 3, AVG(Marks) = (90+70+85)/3 = 81.67
• "CS201": COUNT(*) = 2, COUNT(Marks) = 1, AVG(Marks) = 80/1 = 80.00 <-- Ravi's NULL mark ignored!
• "EC204": COUNT(*) = 2, COUNT(Marks) = 2, AVG(Marks) = (95+75)/2 = 85.00

(c) SELECT Name FROM Student WHERE DeptID NOT IN (SELECT DeptID FROM Student WHERE CGPA < 7.00);
• In Table 7.2, Kabir has CGPA = 6.50 (< 7.00) and DeptID = NULL! So the subquery returns (NULL).
• DeptID NOT IN (NULL) evaluates to UNKNOWN for every student -> Returns 0 rows!

Worked Solution D2 | Bag vs. Set Multiplicities on R = {1,1,1,2,2,3} and S = {1,2,2,2,4}

• (i) R UNION S: {1, 2, 3, 4} (4 rows)
• (ii) R UNION ALL S: 6 + 5 = 11 rows
• (iii) R INTERSECT S: {1, 2} (2 rows)
• (iv) R INTERSECT ALL S: min(3,1)=1 copy of 1; min(2,3)=2 copies of 2 -> {1, 2, 2} (3 rows)
• (v) R EXCEPT S: {3} (1 row)
• (vi) R EXCEPT ALL S: max(0, 3−1)=2 copies of 1; max(0, 2−3)=0 copies of 2; max(0, 1−0)=1 copy of 3 -> {1, 1, 3} (3 rows)!

Solutions to Part D (D3 & D4) and Part E: GATE CS Questions (E1 to E5)

Worked Solutions D3 (LIKE with ESCAPE) & D4 (Conditional Aggregation Report)

```
-- Solution D3 (a): Start with literal "CS_", followed by 2 chars, ending with literal "%"
WHERE Code LIKE 'CS\___\%%' ESCAPE '\'   -- Wait: ends with literal %: 'CS\___\%' ESCAPE '\'
-- Solution D3 (b): Contains at least two 'a' characters anywhere
WHERE Code LIKE '%a%a%'

-- Solution D4: Single-Scan Course Exam & Pass Rate Report
SELECT CourseID,
       COUNT(*) AS TotalEnrolled,
       SUM(CASE WHEN Grade IS NOT NULL AND Grade <> 'F' THEN 1 ELSE 0 END) AS PassedCount,
       SUM(CASE WHEN Marks IS NULL THEN 1 ELSE 0 END) AS PendingExamCount,
       ROUND(
           SUM(CASE WHEN Grade IS NOT NULL AND Grade <> 'F' THEN 1 ELSE 0 END) * 100.0
           / NULLIF(COUNT(Marks), 0), 2
       ) AS PassRatePct
FROM Enrollment
GROUP BY CourseID;
```

Step-by-Step GATE CS Solutions for Questions E1 to E5

GATE Q#	Correct Answer	Complete Step-by-Step Derivation
E1 (NAT)	100; 3 rows	Groups formed by A: A=1 -> COUNT(B)=1, SUM(B)=10; A=2 -> COUNT(B)=2, SUM(B)=50; A=NULL -> COUNT(B)=1, SUM(B)=40. Total output rows = 3; sum of SUM(B) = 10 + 50 + 40 = 100.
E2 (NAT)	6	Notice the algebraic identity: min(m,n) + max(0, m−n) = m for every element! Thus (R INTERSECT ALL S) UNION ALL (R EXCEPT ALL S) reconstructs bag R exactly -> 4 + 2 = 6 rows!
E3 (MCQ)	(B)	WHERE filters rows (CGPA >= 7.0) BEFORE GROUP BY; HAVING filters groups (COUNT(*) >= 2) AFTER GROUP BY.
E4 (MSQ)	(A), (C), (D)	(A) is valid by the SQL:1999 Primary Key FD rule (StudentID is PK). (B) is INVALID because Name is not functionally dependent on DeptID. (C) and (D) are valid.
E5 (NAT)	-0.23	AVG(CGPA) over 5 non-nulls = (8.5+7.5+9.0+6.5+8.0)/5 = 39.5/5 = 7.90. With COALESCE(CGPA, 6.50) over all 6 rows = (39.5+6.5)/6 = 46.0/6 = 7.6667. Difference = 7.67 − 7.90 = -0.23!

Solutions to Part F: Viva & Technical Placement Interview Questions (F1 to F5)

- Answer F1 (WHERE vs. HAVING): WHERE filters individual rows at Stage 2 before grouping and cannot contain aggregate functions. HAVING filters entire groups at Stage 4 after grouping and typically tests aggregate conditions (e.g., HAVING COUNT(*) > 5). A condition on a grouping column (like DeptID = 10) works in both, but placing it in WHERE is much faster because it prunes rows before hashing/sorting.
- Answer F2 (Why DELETE Is Slow & Bloats Disk vs. Instant TRUNCATE): Under MVCC (PostgreSQL), DELETE visits every row, writes a WAL log record, and marks the tuple header xmax as dead without freeing disk space until VACUUM runs. TRUNCATE simply unlinks the physical data files in O(1) time and allocates a fresh empty 8 KB file.
- Answer F3 (Why Prefer UNION ALL Over UNION): Plain UNION forces the database engine to run an expensive Sort or HashAggregate pass over the combined result set to eliminate duplicates. UNION ALL simply streams the second query's rows after the first with zero memory or sorting overhead.
- Answer F4 (Python/JDBC NULL Handling Trap): In Python (psycopg2/sqlite3), SQL NULL maps cleanly to Python None. In Java JDBC, however, primitive rs.getInt("DeptID") returns 0 when the SQL column is NULL! You must call rs.wasNull() or use rs.getObject("DeptID", Integer.class) so NULL maps to a null Integer reference.
- Answer F5 (Finding Duplicate Emails): Write SELECT Email, COUNT(*) FROM Users GROUP BY Email HAVING COUNT(*) > 1; — GROUP BY buckets identical emails together, and HAVING keeps only buckets with 2 or more rows.

Solutions to Part G: Hands-On SQL Labs (Labs 7.1 to 7.3)

Complete SQL Solutions for Labs 7.1, 7.2, and 7.3

```
-- Lab 7.1: Finding Duplicate (Name, City) Pairs and Deleting Duplicates Keeping Lowest StudentID
SELECT Name, City, COUNT(*) AS DupCount, MIN(StudentID) AS KeepID
FROM Student
GROUP BY Name, City
HAVING COUNT(*) > 1;

DELETE FROM Student s1
WHERE EXISTS (
    SELECT 1 FROM Student s2
    WHERE s2.Name = s1.Name
      AND s2.City IS NOT DISTINCT FROM s1.City
      AND s2.StudentID < s1.StudentID
);

-- Lab 7.2: Department KPI Dashboard Query
SELECT COALESCE(d.DeptName, 'Undeclared Department') AS DeptName,
       COUNT(s.StudentID)                            AS StudentCount,
       ROUND(AVG(s.CGPA), 2)                         AS MeanCGPA,
       COALESCE(SUM(s.Scholar), 0.00)                AS TotalScholarshipDisbursed
FROM Student s
LEFT JOIN Department d ON s.DeptID = d.DeptID
GROUP BY d.DeptID, d.DeptName
ORDER BY TotalScholarshipDisbursed DESC;
```

Solutions to Part H: "Spot the Mistake" Diagnostic Challenges (7.1 to 7.3)

Diagnostic Solution 7.1 | Using Aggregate Alias Inside WHERE

Writing SELECT DeptID, AVG(CGPA) AS AvgG FROM Student WHERE AvgG > 8.0 GROUP BY DeptID fails for TWO reasons: (1) WHERE executes at Stage 2 before aggregates (Stage 4) and SELECT aliases (Stage 6) exist, and (2) filtering groups by an aggregate requires HAVING AVG(CGPA) > 8.0!

Diagnostic Solution 7.2 | Unprotected Scalar Addition with Nullable Column

Writing Salary + Bonus produces NULL for every employee whose Bonus is NULL (Trap 6: Contagious NULL arithmetic)! Always wrap nullable numeric columns in COALESCE: Salary + COALESCE(Bonus, 0).

Diagnostic Solution 7.3 | Violating the Single-Value Rule of GROUP BY

Writing SELECT DeptID, Name, AVG(CGPA) FROM Student GROUP BY DeptID is rejected by standard SQL and PostgreSQL because Name is neither aggregated nor functionally dependent on DeptID. Even in legacy MySQL (with ONLY_FULL_GROUP_BY disabled), it returns an arbitrary student's name—NOT the student with the highest/average CGPA!

END OF CHAPTER 7 | Complete DBMS Study Series (Chapters 1 to 7 Ready)

You have now mastered Part I (Chapters 1–4: DBMS Fundamentals, Architecture, Data Models, ER & EER Modeling) and Part II (Chapters 5–7: The Relational Model, Keys, Constraints, Relational Algebra/Calculus, and SQL DDL/DML/Aggregates/3VL)!